Sentinel AML — Project Blueprint & Learning Roadmap

Nov 5, 2026 · @Rupam Biswas

1. Vision

Sentinel is a cloud-native, event-driven Anti-Money-Laundering (AML) Transaction Monitoring System built as Spring Boot microservices on AWS, with a Next.js analyst console. It turns a bank's 48–72 hour, spreadsheet-driven review into sub-second detection, explainable risk-scored alerts and an auditable case workflow.

The project is deliberately designed so that every skill you have learned but not yet used in production — Redis, ActiveMQ, API Gateway, Jenkins, Kubernetes, Terraform, Docker, testing frameworks — solves a real, named problem rather than being bolted on for the resume. Each one has a job, a metric and a screenshot.

North-star outcome (what the demo proves): a transaction posted to the API is validated, normalized to INR, evaluated by the rule engine, scored, de-duplicated and visible in the analyst queue in under 1 second, while a 10,000-transaction bulk load finishes in under 2 minutes — proven with JMeter and Grafana screenshots.

How to use this document

- Sections 2–5 are the *what* and *why*: problems, architecture, tech choices, data and rules.
- Section 6 is the *how*: 10 build phases, each with what to learn, must-know concepts, tasks, a definition of done and a LinkedIn post idea.
- Sections 7–9 are timeline, resume/visibility strategy and risks.
- Tick the task boxes as you go; this doc becomes your project journal.

Decisions locked in

Area	Choice	Note
Backend	Java 21 (LTS), Spring Boot 3.x	Meets the Java 17+ mandate
Architecture	Microservices, event-driven	6 services + gateway
Frontend	Next.js 14+ (App Router, TypeScript)	Analyst console

Area	Choice	Note
Database	MySQL 8 (AWS RDS)	Brief accepts MySQL; one schema per service
Cache	Redis (AWS ElastiCache)	Sliding windows, idempotency, locks
Messaging	ActiveMQ Artemis (Amazon MQ)	Async pipeline between services
Cloud	AWS (EKS, RDS, ElastiCache, Amazon MQ, ECR, S3, Secrets Manager)	Provisioned by Terraform
CI/CD	Jenkins + Docker + Helm	Pipeline to EKS
Testing	JUnit 5, Mockito, Testcontainers, REST Assured, Playwright, JMeter	Unit to load

2. Problem catalogue

Sentinel solves 30 problems: 21 taken directly from the MeridianTrust brief (P1–P21) and 9 industry-grade problems added to make it resume-worthy (X1–X9). Every problem has an ID that the phases in Section 6 reference, so you can always say *which problem this commit solved*.

2.1 Problems from the brief

ID	Problem	Source in brief	How Sentinel solves it	Phase
P1	Suspicious activity found 48–72 h late, after funds can be frozen	Business scenario	Event-driven pipeline: ingest → detect → alert in < 1 s	3, 4
P2	No proper data model for customer → account → transaction	Section A	Normalized MySQL schemas with FKs, Flyway migrations, ERD	2
P3	Bulk loading of KYC, accounts, transactions	Section A	CSV upload endpoint + Spring Batch chunked import	3
P4	Malformed records and broken references	Section A	Bean Validation, FK checks, `ingestion_error` table, rejected-row report	3

ID	Problem	Source in brief	How Sentinel solves it	Phase
P5	Continuous (streaming) transaction arrival	Section A	REST ingest → ActiveMQ queue → detection consumers	3
P6	Large single transactions (≥ $10,000) must be flagged (CTR)	Rule 1	`CTR_THRESHOLD` rule on INR-normalized amount	4
P7	Structuring / smurfing: 3+ txns of $9,000–$9,999 in 24 h	Rule 2	Redis sorted-set sliding window per account	4
P8	Rapid movement / layering: ≥ 80% out within 48 h	Rule 3	Redis window of credits vs. debits per account	4
P9	High-risk / sanctioned jurisdictions and counterparties	Rule 4	Configurable watchlist table, cached in Redis, fuzzy name match	4
P10	Behavioral deviation: daily value > 3× 90-day average	Rule 5	Nightly baseline job + Redis daily counters	4
P11	Repeated round-number / just-below-threshold amounts	Section B	Round-amount rule with modulo + frequency check	4
P12	Analysts cannot see *why* something was flagged	CCO quote	Explanation template + evidence transaction IDs per alert	4
P13	Rules require code redeploy to tune	Section B	`rule_config` table + admin API, hot-reloaded via ActiveMQ topic	5
P14	Alert floods: 50 alerts for one pattern	Section B	De-dup key (customer + rule + window) in Redis, aggregation into one alert	5
P15	No prioritization of alerts	Rule 7	Weighted 0–100 risk score, queue sorted by score	4, 5

ID	Problem	Source in brief	How Sentinel solves it	Phase
P16	Alerts silently deleted	Rule 6	Soft state only; disposition reason + analyst identity mandatory	6
P17	PII exposed in list views	Rule 8	Role-based masking in DTO layer; full view only for authorized roles	6
P18	Multi-currency amounts cannot be compared	Rule 9	`fx_rate` table, normalization to INR at ingestion, Redis-cached rates	3
P19	Unauthorized access to sensitive APIs	NFR Security	Spring Security + JWT (Keycloak/Cognito), RBAC at API layer and gateway	6
P20	No immutable audit trail of state changes	NFR Auditability	Append-only `audit_event` table with hash chain	6
P21	Duplicate or lost alerts under concurrent streams	NFR Concurrency	Idempotency keys, Redis locks, DB unique constraints, transactional outbox	5

2.2 Industry-grade problems we add

These are the problems real banks and fintechs pay senior engineers to solve. Adding them is what moves this from "hackathon project" to "production-style system".

ID	Problem	Why it matters in the real world	How Sentinel solves it	Skill showcased	Phase
X1	Same transaction submitted twice (network retries)	Duplicate postings double-count risk and create false alerts	Idempotency-Key header + Redis SETNX with TTL	Redis	3
X2	A message broker crash loses transactions	Regulators treat lost monitoring as a compliance failure	Transactional outbox, persistent queues, dead-letter queue with replay	ActiveMQ	5
X3	Traffic spikes (salary day, festival sales) overwhelm services	Banks see 5–10× normal volume on peak days	API Gateway rate limiting + Kubernetes HPA autoscaling	API Gateway, K8s	7, 9

ID	Problem	Why it matters in the real world	How Sentinel solves it	Skill showcased	Phase
X4	"Works on my machine" and manual deployments	Manual releases are a top cause of outages	Dockerized services, Jenkins pipeline, Helm deploys	Docker, Jenkins	7, 8
X5	Hand-built cloud environments that cannot be reproduced	Auditors require reproducible, reviewed infrastructure	All AWS infra as Terraform modules with remote state	Terraform	8
X6	No visibility into failures across services	Debugging a 6-service flow without tracing takes hours	Micrometer + Prometheus + Grafana, OpenTelemetry tracing, correlation IDs	Observability	9
X7	Analysts spend hours writing Suspicious Activity Reports	SAR drafting is the most time-consuming analyst task	Auto-generated SAR narrative from alert evidence (template, optional LLM)	Product thinking	10
X8	Laundering rings hidden across many accounts	Layering uses chains of mule accounts	Graph view of counterparty links in the case screen	Full-stack	10
X9	No measurement of rule quality	Compliance teams must justify thresholds to regulators	Rule versioning + metrics: alert volume, false-positive rate, time-to-disposition	Data + dashboards	10

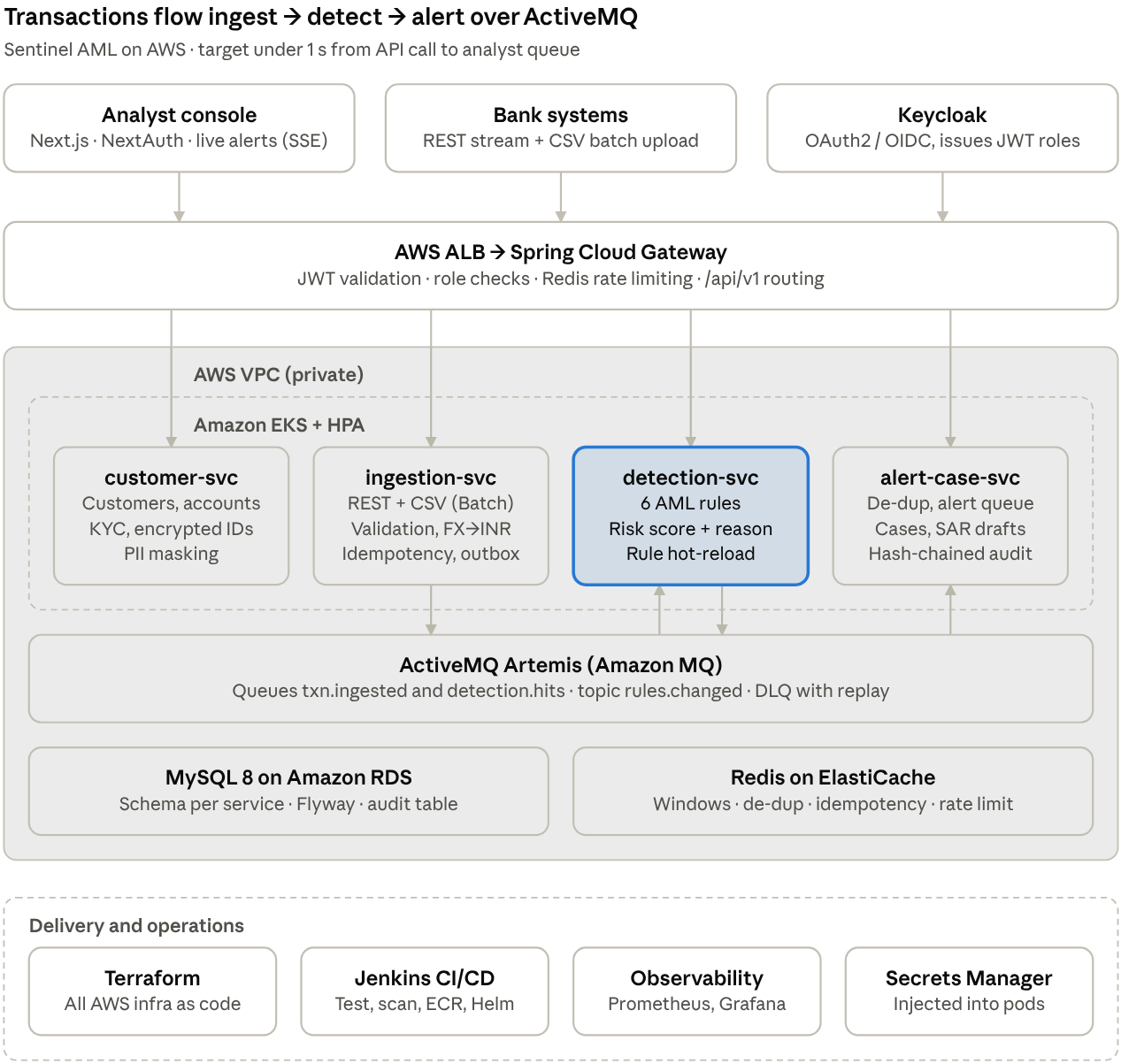
2.3 Scenarios the seed data must reproduce

The synthetic dataset (no real PII, generated with Java Faker or Python Faker) must contain planted cases so the demo always triggers each rule:

1. A customer making four deposits of ₹7.6–8.3 lakh (≈ $9,100–$9,900) within 20 hours → Structuring.
2. A business account receiving ₹50 lakh and wiring ₹42 lakh abroad within 30 hours → Rapid Movement.
3. A transfer to a counterparty in a listed high-risk jurisdiction → High-Risk Jurisdiction.
4. A student account averaging ₹15,000/day suddenly moving ₹2 lakh in one day → Behavioral Deviation.
5. Six transfers of exactly ₹5,00,000 in a week → Round Number.
6. One ₹12 lakh cash deposit → CTR threshold.
7. A mix of the above on one customer → proves aggregation and score stacking.

3. Target architecture

Sentinel is 5 Spring Boot services behind a Spring Cloud Gateway, talking asynchronously through ActiveMQ, with MySQL as the system of record and Redis as the fast detection memory. This is the diagram for LinkedIn post #1.



Sentinel architecture · 5 services behind a gateway, on AWS

The detection service (highlighted) is the heart: it is the one that scales out under load and the one interviewers will ask about.

End-to-end flow of one transaction

1. Bank system calls `POST /api/v1/transactions` with a JWT (client-credentials) and an `Idempotency-Key`.
2. Gateway validates the token, checks the rate limit in Redis and routes to ingestion-service.
3. ingestion-service validates, converts to INR, saves the row and an outbox event in one DB transaction, returns `202 Accepted`.
4. The outbox poller publishes `TransactionIngested` to the `sentinel.txn.ingested` queue.
5. detection-service consumes it, updates Redis windows atomically, runs all enabled rules, computes the score and explanation, publishes `RuleHitsDetected`.
6. alert-case-service de-duplicates, creates or updates an alert, writes an audit event and pushes it to the console via SSE.
7. The analyst opens the alert, reviews evidence, dispositions it or escalates to a case; every step is audited.

Communication rules

- Synchronous REST only for user-facing queries and admin actions; everything on the detection path is asynchronous.
- Every message carries a correlation ID and an event ID; every consumer is idempotent.
- A separate optional notification-service (email/Slack on high-risk alerts) can subscribe to alert events later.

4. Tech stack — each tool, its job, and the problem it solves

Every technology below earns its place by solving a problem ID from Section 2. If you are asked in an interview "why Redis?", the answer is the problem, not the buzzword.

Technology	Exact job in Sentinel	Problems solved	Why this and not something else
Java 21 + Spring Boot 3	All 6 backend services	All	Brief mandate; virtual threads help high-concurrency I/O

Technology	Exact job in Sentinel	Problems solved	Why this and not something else
Spring Data JPA + MySQL 8	System of record for customers, accounts, transactions, alerts, cases, audit	P2, P16, P20	Relational integrity for financial data; brief accepts MySQL
Flyway	Versioned schema migrations per service	P2	Required "migration scripts" deliverable; repeatable on every env
Spring Batch	Chunked, restartable CSV bulk import	P3, P4	Handles 10k+ rows with skip/retry policies and job history
Redis	Sliding windows (sorted sets), daily counters, de-dup keys, idempotency keys, distributed locks, FX + watchlist cache	P7, P8, P10, P14, P18, P21, X1	Detection needs "last 24 h for this account" in microseconds; hitting MySQL per transaction would not stay sub-second
ActiveMQ Artemis (Amazon MQ)	Queues between ingestion → detection → alert; topic for rule-config reload; DLQ	P1, P5, P13, X2	You already know it; JMS gives persistent delivery, redelivery and DLQ out of the box; Amazon MQ is managed on AWS
Spring Cloud Gateway	Single entry point: JWT validation, routing, rate limiting (Redis), CORS	P19, X3	Code-level gateway you control and can demo; AWS ALB sits in front of it
Keycloak (or AWS Cognito)	Identity provider issuing JWTs with roles	P19, P17	Real OAuth2/OIDC instead of hand-rolled login; Keycloak runs locally for free

Technology	Exact job in Sentinel	Problems solved	Why this and not something else
Resilience4j	Circuit breakers and retries between services	X2, X3	Stops one slow service cascading into a full outage
Docker + Docker Compose	Containerize every service; one-command local stack	X4	Same image runs locally, in Jenkins and on EKS
Jenkins	CI/CD: build, test, scan, image push, Helm deploy	X4	You know it; declarative pipeline is a strong resume artifact
Kubernetes (AWS EKS) + Helm	Run, scale and heal services; HPA autoscaling	X3, X4	Industry standard; HPA screenshot during JMeter test is a great visual
Terraform	VPC, EKS, RDS, ElastiCache, Amazon MQ, ECR, S3, IAM, Secrets Manager	X5	Reproducible infra; reviewers can read the whole cloud in Git
AWS Secrets Manager + External Secrets Operator	Inject DB/MQ passwords into pods	NFR Security	Brief forbids hard-coded secrets
springdoc-openapi	Swagger UI + OpenAPI spec per service, aggregated at gateway	NFR API	Required deliverable
Micrometer, Prometheus, Grafana, OpenTelemetry (+ Tempo/Jaeger), Loki	Metrics, dashboards, tracing, logs	X6, P1 proof	Proves latency claims with graphs, not words
JUnit 5 + Mockito	Unit tests for every rule	Brief deliverable	Required

Technology	Exact job in Sentinel	Problems solved	Why this and not something else
Testcontainers	Integration tests against real MySQL, Redis, ActiveMQ in Docker	P21, X2	Catches bugs mocks hide; strong signal of maturity
REST Assured	API contract tests	NFR API	Verifies status codes and error JSON
Playwright	End-to-end UI tests of the analyst flow	Frontend quality	Modern, fast, works great with Next.js
JMeter	Bulk-load and streaming load tests	Performance NFR	Your planned LinkedIn proof; produces HTML reports
SonarQube + Trivy	Code quality and container vulnerability scans in Jenkins	Security	Banks require security gates in pipelines
Next.js + TypeScript + Tailwind + shadcn/ui + TanStack Query + Recharts + React Flow	Analyst console, dashboards, graph view	P12, P15, X7–X9	Fast to build, SSR, good component ecosystem

4.1 MySQL notes (vs. the Postgres preference)

The brief prefers PostgreSQL but accepts MySQL, so MySQL is fine. Things to handle consciously:

- Use `DECIMAL(19,4)` for money, never `FLOAT` / `DOUBLE`.
- Use InnoDB, `utf8mb4`, and `DATETIME(6)` stored in UTC.
- MySQL has no native append-only tables: enforce audit immutability with a DB user that has only `INSERT, SELECT` on `audit_event`, plus a hash chain.
- JSON columns work well for alert evidence and rule parameters.
- Mention in the README *why* you chose MySQL (team familiarity, RDS support, brief allows it) — reviewers like reasoned decisions.

5. Data model and detection design

Each service owns its own MySQL schema (database-per-service); services never join across schemas — they share data through events. All money is stored twice: original amount + currency, and normalized `amount_inr`.

5.1 Core tables by service

Service (schema)	Tables	Key columns
customer-service (`customer_db`)	`customer`, `account`, `kyc_document`	customer: id, type (RETAIL/BUSINESS), full_name, id_number (encrypted), dob, nationality, risk_rating, onboarded_at · account: id, customer_id FK, type, currency, opened_at, risk_rating, status
ingestion-service (`ingestion_db`)	`transaction`, `fx_rate`, `ingestion_batch`, `ingestion_error`, `outbox_event`	transaction: id, external_ref (unique), account_id, direction (CREDIT/DEBIT), amount, currency, amount_inr, counterparty_name, counterparty_account, counterparty_country, channel, txn_timestamp, jurisdiction
detection-service (`detection_db`)	`rule_config`, `rule_version`, `watchlist_entry`, `customer_baseline`, `rule_hit`	rule_config: rule_code, enabled, weight, params JSON, version · baseline: customer_id, avg_daily_value_90d, avg_daily_count_90d, computed_at

Service (schema)	Tables	Key columns
alert-case-service (`case_db`)	`alert`, `alert_evidence`, `case`, `case_alert`, `case_note`, `audit_event`	alert: id, customer_id, rule_codes, risk_score, status, dedup_key (unique), explanation, first_seen, last_seen, hit_count, disposition, disposition_reason, disposed_by · audit_event: id, entity, entity_id, from_state, to_state, actor, timestamp, prev_hash, hash
reporting (inside alert-case or separate)	`sar_draft`, `rule_metric_daily`	Extensions X7, X9

State machines (enforced in code and audited):

- **Alert:** `NEW → IN_REVIEW → ESCALATED_TO_CASE | CLOSED_FALSE_POSITIVE | CLOSED_NO_ACTION`. Never deleted (P16).
- **Case:** `OPEN → INVESTIGATING → PENDING_APPROVAL → SAR_FILED | CLOSED`. Closing requires reason + analyst ID; SAR filing requires a supervisor (maker-checker).

5.2 Roles (RBAC)

Role	Can do
`ANALYST`	View masked queue, open alert detail, add notes, disposition alerts, create cases
`SENIOR_ANALYST`	All of the above + see full PII, approve case closure
`COMPLIANCE_OFFICER`	Approve SAR filing, view dashboards, view audit trail
`RULE_ADMIN`	Create/tune/toggle rules, manage watchlists and FX rates
`INGEST_SYSTEM`	Machine role (client credentials) for posting transactions

5.3 Detection rules

All thresholds live in `rule_config.params` and can be changed through `PUT /api/v1/rules/{code}` without redeployment.

Rule code	Business rule	Logic	Redis structure	Default weight
`CTR_THRESHOLD`	Rule 1	`amount_inr ≥ threshold` (₹8.3 lakh ≈ $10,000 via FX table)	none	30
`STRUCTURING`	Rule 2	≥ 3 txns in [$9,000, $9,999] in 24 h on one account	ZSET `win:struct:{acct}` scored by timestamp; ZREMRANGEBYSCORE then ZCOUNT	70
`RAPID_MOVEMENT`	Rule 3	Sum of debits in 48 h after a credit ≥ 80% of credit	ZSETs `win:cr:{acct}`, `win:dr:{acct}`	75
`HIGH_RISK_JURISDICTION`	Rule 4	Country or counterparty on watchlist, any amount	SET `watch:countries`, HASH `watch:names` (+ Jaro-Winkler fuzzy match)	90
`BEHAVIORAL_DEVIATION`	Rule 5	Today's value > 3× 90-day average daily value	HASH `day:{cust}:{yyyymmdd}`; baseline from nightly job	55
`ROUND_AMOUNT`	Section B	≥ N txns in 7 days where amount % 1000 = 0 or within 2% below a threshold	ZSET `win:round:{acct}`	40

Risk score (Rule 7). Combine triggered rule weights so that more evidence pushes toward 100 but never past it, then adjust for customer risk:

```
\text{score} = \min\left(100,\; \left(1 - \prod_{i}\left(1 - \frac{w_i}
{100}\right)\right) \times 100 \times m_{\text{customer}}\right)
```

where m = 1.0 (LOW), 1.15 (MEDIUM), 1.3 (HIGH customer risk rating). Example: Structuring (70) + Round (40) on a MEDIUM customer = (1 − 0.3 × 0.6) × 100 × 1.15 ≈ 94.

Explanation (P12). Every rule returns a template filled with facts, e.g. *"Account XX-4421 received 4 cash deposits between ₹7.6L and ₹8.3L (each just under the ₹8.3L reporting threshold) within 19 h 40 m. Evidence: TXN-1021, TXN-1033, TXN-1040, TXN-1052."*

De-duplication (P14). `dedup_key = customerId : ruleCode : windowBucket`. If the key exists in Redis (SETNX fails), the new hit is *appended* as evidence to the existing open alert, `hit_count` and `last_seen` update, and the score is recomputed — instead of a new alert. A unique DB constraint on `dedup_key` is the safety net (P21).

6. Build plan — 11 phases, learning-driven

The build runs in 11 phases (0–10), each ending in something demoable and postable. Every phase lists: the problems it solves, what to learn, the must-know concepts for interviews, tasks, a definition of done (DoD), and a LinkedIn post with its diagram/screenshot.

Working rules for every phase

- One Git branch per phase, merged via PR with a description that cites problem IDs (e.g. "Solves P7, P14").
- Write tests in the same phase as the code — never "testing later".
- Keep an `ADR/` folder: one short Architecture Decision Record per big choice (why MySQL, why ActiveMQ, why database-per-service). Interviewers love these.
- Learn just enough to build the step; depth comes from building.

Phase 0 — Design and kickoff (week 1)

Solves: sets up the story for all problems. **Skills:** system design, diagramming.

Learn: C4 model (context, container diagrams); basics of AML (placement, layering, integration; CTR vs SAR; FIU-IND in India, FinCEN in the US); microservice decomposition by business capability.

Must know: why microservices (independent scaling of detection vs. UI) and their costs (distributed data, eventual consistency); what a bounded context is; synchronous vs. asynchronous communication.

- [ ] Create GitHub monorepo `sentinel-aml` with folders `services/`, `frontend/`, `infra/`, `deploy/`, `docs/`, `load-tests/`
- [ ] Draw the C4 context and container diagrams (Excalidraw or draw.io) — based on Section 3
- [ ] Write ADR-001 (microservices), ADR-002 (MySQL), ADR-003 (ActiveMQ)
- [ ] Write the README skeleton with the problem statement

DoD: repo public, diagrams committed in `docs/`. **LinkedIn post #1:** "I'm building a real-time AML transaction monitoring system in public" + architecture diagram + the 48–72 h problem and the < 1 s target.

Phase 1 — Local platform and service skeletons (weeks 2–3)

Solves: foundation for X4. **Skills:** Docker, Docker Compose, Maven multi-module, Spring Cloud Gateway (basic).

Learn: Dockerfile best practices (multi-stage builds, non-root user, layered jars); Compose networks, volumes, healthchecks; Spring Boot profiles; Maven parent POM and BOMs.

Must know: image vs. container; layer caching; why `depends_on` doesn't wait for readiness (use healthchecks); 12-factor config via environment variables.

- [ ] Maven parent + modules: `common-lib`, `gateway-service`, `customer-service`, `ingestion-service`, `detection-service`, `alert-case-service`, `notification-service` (optional)
- [ ] `common-lib`: error response format (RFC 7807 Problem Details), correlation-ID filter, event DTOs
- [ ] `docker-compose.yml`: MySQL 8, Redis 7, ActiveMQ Artemis, Keycloak, Adminer, Redis Insight
- [ ] Each service: `/actuator/health`, Swagger UI, a multi-stage Dockerfile
- [ ] Gateway routes `/api/v1/customers/**`, `/api/v1/transactions/**` etc.
- [ ] `.env.example` — no secrets in Git (add `gitleaks` pre-commit hook)

DoD: `docker compose up` brings up 12 containers healthy. **Post #2:** "One command, 12 containers" screenshot of `docker ps` + Compose diagram + 3 Dockerfile lessons.

Phase 2 — Data model and customer-service (weeks 3–4)

Solves: P2. **Skills:** JPA, Flyway, MySQL, JUnit, Testcontainers.

Learn: JPA relationships and fetch types; N+1 problem; Flyway naming (`V1__init.sql`); MySQL indexing (composite indexes on `account_id, txn_timestamp`); `DECIMAL` for money; Testcontainers lifecycle.

Must know: LAZY vs. EAGER; optimistic locking with `@Version`; why database-per-service; ACID and isolation levels (MySQL default REPEATABLE READ).

- [ ] Flyway migrations for customer and account tables; generate ERD (MySQL Workbench or dbdiagram.io)
- [ ] CRUD + search APIs: `POST/GET /api/v1/customers`, `/api/v1/customers/{id}/accounts`
- [ ] Field-level encryption for `id_number` (JPA `AttributeConverter` + AES-GCM, key from env)
- [ ] Synthetic data generator (Java Faker): 5,000 customers, 8,000 accounts
- [ ] Repository tests with Testcontainers MySQL; controller tests with MockMvc

DoD: ERD in `docs/`, 80%+ coverage on service layer. **Post #3:** the ERD + "why I store money as DECIMAL and encrypt ID numbers".

Phase 3 — Ingestion: bulk, streaming, validation, FX (weeks 5–6)

Solves: P3, P4, P5, P18, X1. **Skills:** Spring Batch, ActiveMQ (producer), Redis (idempotency, cache), validation.

Learn: Spring Batch (job, step, chunk, reader/processor/writer, skip policy); JMS with Spring (`JmsTemplate`, persistent delivery); Jakarta Bean Validation and custom validators; Redis `SET NX EX`; Spring Cache with Redis.

Must know: queue vs. topic; at-least-once delivery; idempotency and why retries need it; transactional outbox pattern (the DB write and the message publish must not diverge).

- [ ] `POST /api/v1/transactions` (single, streaming) with `Idempotency-Key` header → 202 Accepted
- [ ] `POST /api/v1/ingestion/batches` (multipart CSV) for customers, accounts, transactions → Spring Batch job, `GET .../batches/{id}` for status and error report
- [ ] Validation: required fields, positive amounts, ISO currency/country codes, account exists (call customer-service or cached account registry)
- [ ] Failed rows → `ingestion_error` with reason; never silently dropped
- [ ] `fx_rate` table + admin API; normalize to `amount_inr` at ingestion; rates cached in Redis

- [ ] Outbox table + poller publishing `TransactionIngested` to `sentinel.txn.ingested` queue
- [ ] Seed CSVs with the planted scenarios from Section 2.3 + 10,000 background transactions

DoD: 10k-row CSV imports, errors reported, duplicate POST returns the original response. **Post #4:** sequence diagram of the ingestion flow + "what idempotency keys taught me".

Phase 4 — Detection engine and risk scoring (weeks 7–9)

Solves: P1, P6–P12, P15. **Skills:** ActiveMQ (consumer), Redis data structures, design patterns, JUnit + Mockito.

Learn: Strategy and Chain-of-Responsibility patterns; Redis sorted sets, hashes, pipelines and Lua scripts (atomic window updates); `@JmsListener` concurrency; scheduling with `@Scheduled` / ShedLock for the nightly baseline job.

Must know: sliding vs. tumbling windows; time complexity of ZADD/ZCOUNT (O(log n)); why atomicity matters for windows under concurrency; how to unit test time (inject a `Clock`).

- [ ] `DetectionRule` interface: `evaluate(TransactionContext) → Optional<RuleHit>`; one class per rule
- [ ] Rule registry that loads enabled rules + params from `rule_config`
- [ ] Implement the 6 rules from Section 5.3 with Redis windows (Lua for atomic add-trim-count)
- [ ] Nightly `BaselineJob` computing 90-day average per customer
- [ ] Risk scorer + explanation builder; publish `RuleHitsDetected` to `sentinel.detection.hits`
- [ ] Watchlist: OFAC/UN-style synthetic list + FATF-style high-risk country list (synthetic), fuzzy name matching
- [ ] JUnit + Mockito tests: for every rule, a "fires", a "does not fire", and an edge case (exactly 3 txns, exactly 80%, boundary at 23 h 59 m)

DoD: all planted scenarios trigger the right rules; ~40+ rule tests green. **Post #5:** rule engine class diagram + a sample explanation text + "testing time-window logic with an injected Clock".

Phase 5 — Alerts, de-dup, hot-reload, concurrency safety (weeks 10–11)

Solves: P13, P14, P15, P21, X2. **Skills:** ActiveMQ (topics, DLQ, redelivery), Redis (locks), concurrency.

Learn: Artemis address/queue model, redelivery delay and DLQ config; JMS topics for broadcast; Redisson distributed locks (or SETNX + TTL); DB unique constraints as last line of defense; `ExecutorService` / virtual threads.

Must know: exactly-once is a myth — build at-least-once + idempotent consumers; race conditions and how to reproduce them in tests; poison messages.

- [ ] alert-case-service consumes hits, applies de-dup key logic, creates or updates alerts
- [ ] `GET /api/v1/alerts?sort=riskScore,desc&status=NEW` with pagination
- [ ] Rule admin API: `GET/PUT /api/v1/rules/{code}` (toggle, weight, params) → new `rule_version` row → publish on `sentinel.rules.changed` topic → every detection pod reloads in memory
- [ ] DLQ for messages failing 5 times + `POST /api/v1/admin/dlq/replay`
- [ ] Concurrency test: 50 threads posting overlapping structuring transactions → assert exactly one alert per pattern (Testcontainers)

DoD: rule threshold changed via API takes effect in < 5 s without restart; concurrency test green. **Post #6:** "How I stopped 50 duplicate alerts" with a before/after diagram of the de-dup flow.

Phase 6 — Case management, security, audit, PII masking (weeks 12–13)

Solves: P16, P17, P19, P20. **Skills:** Spring Security, OAuth2/JWT, Keycloak, API Gateway security.

Learn: OAuth2 flows (authorization code + PKCE for the UI, client credentials for `INGEST_SYSTEM`); JWT structure and validation; Spring Security resource server, `@PreAuthorize`; Keycloak realms, clients, roles; Spring Data JPA auditing; Jackson views or DTO mappers (MapStruct) for masking.

Must know: authentication vs. authorization; why RBAC must be enforced in the API not the UI; token expiry/refresh; maker-checker (four-eyes) principle; why audit logs must be tamper-evident.

- [ ] Keycloak realm `sentinel` with the 5 roles from Section 5.2 and test users
- [ ] Gateway validates JWT; each service re-validates and enforces `@PreAuthorize` (defense in depth)

- [ ] Alert workflow APIs: assign, start review, disposition (reason + analyst required), escalate to case
- [ ] Case APIs: create, link alerts, notes, attach evidence, submit for approval, approve/reject (different user than submitter)
- [ ] `DELETE` on alerts returns 405 — by design (P16)
- [ ] Masking: list endpoints return `R****h S***a`, `XXXX-XXXX-4521`; detail endpoint returns full PII only to `SENIOR_ANALYST`+
- [ ] Audit: every state transition writes `audit_event` with `prev_hash` → `hash` (SHA-256) chain; `GET /api/v1/audit/verify` checks the chain
- [ ] Security tests: each endpoint × each role → expected 200/403 (parameterized JUnit)

DoD: full alert → case → closure flow works via Swagger with role switching. **Post #7:** state machine diagram + "how I made the audit log tamper-evident with a hash chain".

Phase 7 — Next.js analyst console + gateway hardening (weeks 14–16)

Solves: P12, P15, P17 (UI side), X3 (rate limiting). **Skills:** Next.js, TypeScript, Playwright, Spring Cloud Gateway.

Learn: Next.js App Router, server vs. client components; NextAuth.js with Keycloak provider; TanStack Query; Recharts; Server-Sent Events (SSE) for live alerts; Gateway `RequestRateLimiter` with Redis token bucket.

Must know: CORS; storing tokens safely (httpOnly cookies, not localStorage); token-bucket algorithm; why rate limit per client ID rather than per IP.

- [ ] Login via Keycloak; role-aware navigation
- [ ] **Alert queue:** sortable by risk score, filter by rule/status, masked PII, live updates via SSE
- [ ] **Alert detail:** explanation, evidence transactions, customer timeline chart
- [ ] **Risk heatmap:** alert count by region × rule (brief Section C)
- [ ] **Case view:** notes, linked alerts, disposition form, approval
- [ ] **Rule admin page:** toggle and tune thresholds live
- [ ] Gateway: rate limit 100 req/s per client, 429 with JSON error; request size limits
- [ ] Playwright E2E: login → open top alert → escalate → close case

DoD: an analyst can complete the full workflow in the UI. **Post #8:** 30-second screen recording of a live alert appearing + the UI architecture.

Phase 8 — AWS infrastructure, CI/CD and Kubernetes (weeks 17–19)

Solves: X4, X5, security NFR (secrets). **Skills:** Terraform, AWS, Jenkins, Kubernetes, Helm.

Learn: Terraform (providers, modules, variables, outputs, remote state in S3 with DynamoDB lock, `plan` / `apply` / `destroy`); AWS VPC (public/private subnets, NAT), EKS, RDS MySQL, ElastiCache Redis, Amazon MQ (ActiveMQ engine), ECR, IAM roles for service accounts (IRSA), Secrets Manager; Kubernetes (Deployment, Service, Ingress, ConfigMap, Secret, probes, resource requests/limits, HPA); Helm charts; Jenkins declarative pipeline, shared libraries, credentials binding.

Must know: Terraform state and why it must be locked; why databases go in private subnets; liveness vs. readiness probes; rolling update vs. blue-green; least-privilege IAM.

- [ ] `infra/terraform/modules/{vpc,eks,rds,elasticache,amazonmq,ecr,secrets}` + `envs/dev`
- [ ] AWS Load Balancer Controller + Ingress → gateway service
- [ ] External Secrets Operator syncing Secrets Manager → K8s Secrets
- [ ] One Helm chart per service (or one umbrella chart with values per service)
- [ ] Jenkins (on EC2 or as a K8s pod) pipeline: checkout → `mvn verify` (unit + Testcontainers) → SonarQube gate → Docker build → Trivy scan → push to ECR → `helm upgrade --install` → smoke test
- [ ] Next.js deployed as a container on EKS (or AWS Amplify if simpler)
- [ ] `terraform destroy` script and a cost-alarm budget (see Section 9)

DoD: a push to `main` deploys to EKS automatically. **Post #9:** Terraform module diagram + Jenkins Blue Ocean pipeline screenshot (all green).

Phase 9 — Observability and performance testing (weeks 20–21)

Solves: X3, X6, performance NFR (10k txns < 2 min, sub-second streaming), P1 proof. **Skills:** JMeter, Prometheus, Grafana, OpenTelemetry, K8s HPA.

Learn: Micrometer timers and percentiles; Prometheus scraping (kube-prometheus-stack); Grafana dashboards; OpenTelemetry Java agent + Tempo/Jaeger; Loki for logs; JMeter thread groups, CSV data sets, assertions, HTML dashboard report, non-GUI runs.

Must know: p50 vs. p95 vs. p99 latency; throughput vs. latency; Little's Law; why to load test from outside the cluster; finding a bottleneck (DB pool, consumer concurrency, Redis round trips).

- [ ] Custom metrics: `sentinel_txn_ingested_total`, `sentinel_detection_latency_seconds`, `sentinel_alerts_created_total{rule}`, end-to-end latency (ingest → alert)
- [ ] Grafana dashboard: throughput, p95 latency, queue depth, alerts per rule, pod count
- [ ] Distributed trace from gateway → ingestion → detection → alert visible in one view
- [ ] JMeter plan A (bulk): upload 10k-row CSV, measure job completion time (target < 2 min)
- [ ] JMeter plan B (stream): ramp to 200 TPS for 10 min, assert p95 end-to-end < 1 s
- [ ] Trigger HPA scale-out of detection-service during plan B; capture pods going 2 → 6
- [ ] Tune: JMS consumer concurrency, Hikari pool, Redis pipelining; record before/after numbers

DoD: published benchmark table in README with real numbers. **Post #10 (your flagship):** JMeter report + Grafana dashboard + HPA screenshots + "what I tuned and the before/after p95".

Phase 10 — Extensions and launch (weeks 22–24)

Solves: X7, X8, X9 + final deliverables. **Skills:** product thinking, data viz, optional LLM integration.

Learn: SAR structure (who, what, when, where, why, how); React Flow for graph visualization; simple statistics for false-positive rate; (optional) calling an LLM API with strict prompting and human-in-the-loop review.

Must know: why automated SAR drafts must be reviewed by a human; precision vs. recall in alerting; A/B comparing rule versions on replayed data.

- [ ] SAR draft generator: template narrative from alert evidence (LLM rewrite optional, clearly labelled draft)
- [ ] Network graph in case view: accounts and counterparties within 2 hops
- [ ] Analyst productivity dashboard: alert volume trend, false-positive rate per rule, average time-to-disposition
- [ ] Rule versioning: replay last 30 days against v1 vs. v2 thresholds, compare alert counts
- [ ] Final README: architecture, setup (local + AWS), rule configuration, benchmarks, ADRs, screenshots
- [ ] Record 5–7 min demo video: ingestion → detection → alert → case → SAR draft → dashboards

DoD: all brief deliverables ticked (Section 8.3). **Post #11:** demo video + full architecture + "24 weeks, 30 problems, 1 system" retrospective.

Testing strategy across all phases

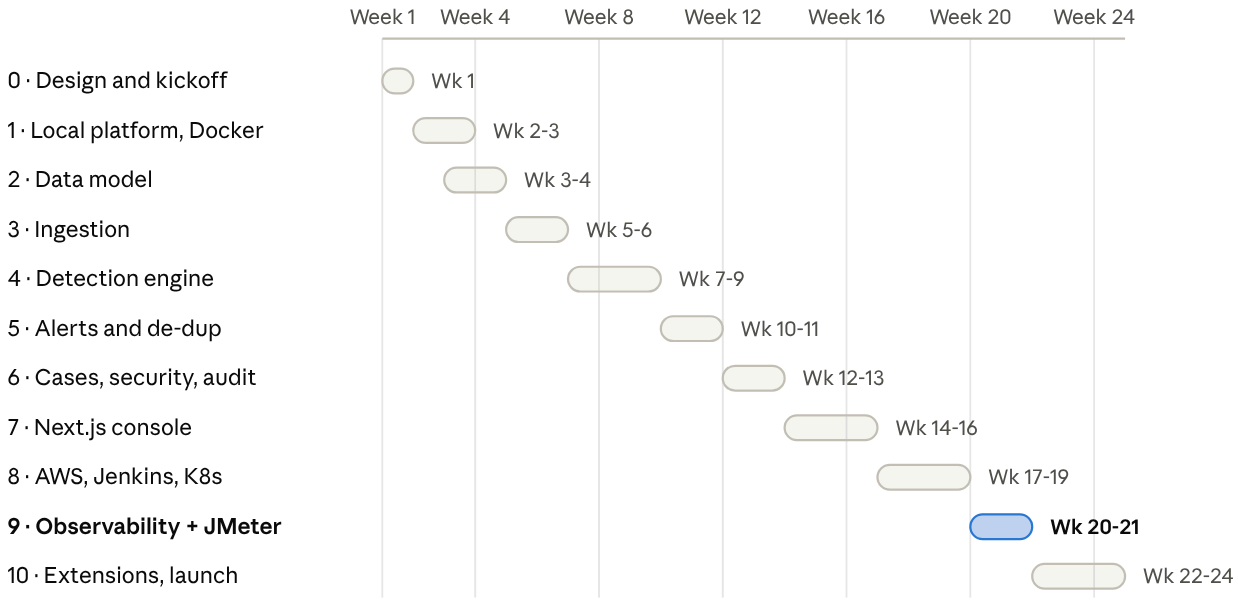
Layer	Tool	What is tested	Target
Unit	JUnit 5 + Mockito + AssertJ	Every rule, scorer, explainer, de-dup logic, state machines	≥ 85% on detection and alert modules
Integration	Testcontainers (MySQL, Redis, Artemis)	Repositories, Lua scripts, JMS flows, concurrency	Every service
Contract/API	REST Assured	Status codes, error JSON, RBAC per role	Every endpoint
E2E	Playwright	Analyst workflow in UI	3 critical journeys
Load	JMeter	Bulk + streaming NFRs	Numbers in README
Security	Trivy, SonarQube, OWASP Dependency-Check	Image CVEs, code smells, vulnerable libs	No HIGH/CRITICAL

7. Timeline

At about 16 hours a week (5 weekdays × 1.5 h + 2 weekend days × 4.5 h), the full build takes roughly 24 weeks and 380 hours; the core system (Phases 0–6) is demoable by week 13.

Core system done by week 13; flagship benchmarks in weeks 20-21

11 phases over 24 weeks at about 16 hours a week



Build timeline · 11 phases, 24 weeks

Phase 9 is highlighted because its JMeter and Grafana results are your strongest LinkedIn and interview material. Phases 1 and 2 overlap in week 3 on purpose: start the schema while Compose stabilizes.

Weekly rhythm

Slot	Time	Use it for
Mon–Tue	1–2 h each	Learning: docs, one tutorial, a spike in a scratch project
Wed–Thu	1–2 h each	Small build tasks: one endpoint, one rule, one test class
Fri	1 h	Tests, refactor, update this doc's checkboxes
Saturday	4–5 h	Deep build block: the hardest task of the week
Sunday	4–5 h	Finish, integrate, draw the diagram, draft the LinkedIn post

If you fall behind, protect the order of phases rather than the dates: an honest week-30 finish with real benchmarks beats a rushed week-24 one.

8. Real-world impact, resume and visibility

The story you tell everywhere is one sentence: *"I built an event-driven AML monitoring platform that cut detection time from 48–72 hours to under a second, with explainable alerts, an auditable case workflow, and fully automated AWS infrastructure."*

8.1 Impact on the real customer world

Stakeholder	Before (the brief)	After Sentinel
Bank / CCO	Regulator flagged monitoring as manual and slow; fines and licence risk	Near-real-time monitoring, documented rules, immutable audit trail to show regulators
Compliance analysts	Nightly spreadsheets, no prioritization, no explanation	Risk-sorted queue, plain-language reasons, evidence in one click, SAR drafts
Honest customers	Blanket manual reviews, delayed payments	De-duplicated, tuned rules mean fewer false-positive holds
Society / FIU	Illicit funds moved before anyone noticed	Funds can be frozen and reported inside the legal window
Engineering / IT	Hand-built servers, manual releases	Terraform, CI/CD, autoscaling, dashboards — reproducible and observable

The same architecture also applies to card fraud detection, UPI transaction monitoring, insurance claims fraud and trade surveillance — mention this in interviews to show transferable thinking.

8.2 Resume bullets (fill real numbers after Phase 9)

Project title: **Sentinel — Real-Time AML Transaction Monitoring Platform** (Java 21, Spring Boot, Next.js, MySQL, Redis, ActiveMQ, AWS EKS, Terraform, Jenkins)

- Architected an event-driven microservice platform (6 Spring Boot services behind Spring Cloud Gateway) that detects money-laundering typologies in **< X ms p95**, replacing a 48–72 hour batch review process.

- Built a configurable rule engine for 6 AML typologies (structuring, rapid fund movement, sanctions, behavioral deviation, round-amount, CTR) using **Redis sorted-set sliding windows and Lua scripts**; thresholds hot-reload via ActiveMQ topics with zero redeploys.
- Eliminated duplicate and lost alerts under concurrent load using **idempotency keys, transactional outbox, DLQ replay and distributed locks**, verified by 50-thread Testcontainers tests.
- Processed **10,000 transactions in X s** (bulk) and sustained **Y TPS** (streaming) in JMeter tests, with Kubernetes HPA scaling detection pods 2 → 6.
- Provisioned AWS (VPC, EKS, RDS MySQL, ElastiCache, Amazon MQ, Secrets Manager) with **Terraform modules**; built a **Jenkins pipeline** with SonarQube and Trivy gates deploying Helm charts to EKS.
- Secured APIs with **OAuth2/JWT (Keycloak) RBAC**, role-based PII masking, AES-encrypted ID numbers and a **hash-chained immutable audit log**.
- Delivered a Next.js analyst console with live alert streaming (SSE), risk heatmap, case workflow, network graph and auto-drafted SAR narratives.

8.3 Brief deliverables checklist

- [ ] Working Spring Boot application in Git
- [ ] Schema/ERD + Flyway migration scripts
- [ ] Synthetic dataset covering ≥ 3 typologies (we cover 6)
- [ ] OpenAPI/Swagger documentation
- [ ] Unit tests for detection rules
- [ ] README: architecture, setup, rule configuration
- [ ] Demo video: ingestion → detection → alert → case disposition

8.4 LinkedIn and visibility plan

- **Cadence:** one post per phase (11 posts over ~24 weeks) plus short "TIL" posts when something breaks and you fix it — failure stories get the most engagement.
- **Post formula:** hook (a problem or number) → diagram or screenshot → 3 things learned → link to repo/commit → one question to readers.
- **Visual assets to capture as you go:** C4 diagrams, ERD, sequence diagrams, rule class diagram, state machine, `docker ps`, Swagger UI, Jenkins pipeline, Terraform graph, K8s pods scaling, Grafana dashboards, JMeter HTML report, distributed trace, UI

recording.

- **Hashtags:** #Java #SpringBoot #Microservices #AWS #Kubernetes #Terraform #DevOps #FinTech #AML #BuildInPublic.
- **Long-form:** after Phases 5 and 9, write a Medium/Hashnode/dev.to article ("Detecting structuring with Redis sorted sets", "Load testing an event-driven system to 200 TPS") and link it from LinkedIn and the README.
- **GitHub polish:** pinned repo, badges (build, coverage), screenshots in README, GitHub Projects board showing phases — recruiters do click through.

8.5 Roles this targets

Backend Engineer (Java/Spring) at fintech and banks, Full-Stack Engineer (Java + React/Next.js), Platform/DevOps Engineer, and SDE-2 roles where system design is assessed. The project gives you a ready answer to the system design, concurrency, "tell me about a hard bug" and "how do you test" interview questions.

9. Risks, scope cuts and AWS cost control

The biggest risk is not technical — it is losing momentum over 24 weeks. Protect the core (Phases 0–6) and treat everything after as upgrades.

Risk	Likelihood	Mitigation
AWS bill surprises (EKS control plane ≈ $0.10/h, NAT gateway, RDS, Amazon MQ)	High	Develop locally on Docker Compose / kind until Phase 8; bring AWS up only for demo and load-test weekends; `terraform destroy` afterwards; AWS Budgets alarm at $20 and $50; use smallest instance classes and a single NAT
Scope creep from extensions	High	Extensions only after Phase 9 benchmarks are done
Losing a week to a hard bug	Medium	Time-box to 2 sessions, then simplify and write it down — it becomes a "TIL" post

Risk	Likelihood	Mitigation
Too many services to run on a laptop	Medium	Give each service `-Xmx256m`; run only the services you are working on; 16 GB RAM recommended
Amazon MQ cost/complexity	Medium	Fallback: run ActiveMQ Artemis as a StatefulSet on EKS (still shows K8s skill)
Burnout	Medium	Take a light catch-up week whenever you need one and shift the timeline; skip a LinkedIn post rather than a rest day

Scope cut order if time runs short (cut from the bottom first): rule A/B replay → network graph → SAR draft → Loki/Tempo (keep Prometheus + Grafana) → notification-service → Amazon MQ (self-host Artemis). Never cut: the 6 rules, tests, de-dup, audit, RBAC, Terraform, Jenkins, JMeter results.

Cheaper AWS alternative if budget is tight: run k3s on a single EC2 `t3.large` spot instance provisioned by Terraform, with MySQL/Redis/Artemis in-cluster. You still demonstrate Terraform, Kubernetes, Helm and Jenkins; mention EKS as the production target in the README.

Open questions to settle before Phase 1

- Will you use Keycloak (free, local, richer to show) or AWS Cognito (managed)? Recommendation: Keycloak.
- Monorepo or one repo per service? Recommendation: monorepo for a portfolio — one link, easier for recruiters.
- Jenkins hosted where? Recommendation: Docker locally for Phases 1–7, then a small EC2 or in-cluster for Phase 8.